

Problem Set 6: Probabilistic Models and Reinforcement Learning

This assignment requires a working IPython Notebook installation, which you should already have. If not, please refer to the instructions in the previous problem sets.

Submission Guidelines

You need to submit **the output of the notebook as PDF** and **an additional .py file** on Gradescope.

For the .py file, Here are the guidelines :

1. The .py file should contain ONLY your typed codes. (Do not include any other code apart from what you coded for the assignment)
2. .py should NOT contain any written answers. Only code written by you.
3. If your typed code falls under a function definition thats predefined by us, Only include your typed code and nothing else.
4. For each cell block within colab/jupyter that you typed your code in, Add 2 new lines ("
") before pasting your typed code in the .py file.

The assignment is graded on completion. You will also need to **answer** whether you attempted the question or not in Gradescope. It is extremely important that you complete all assignments (both the programming and written questions), as they will prepare you for quizzes and exams.

Total: 125 points (+20 points bonus).

1 Linear Discriminant Analysis (LDA) [50 pts]

In this exercise, we build a linear discriminant analysis (LDA) classifier for the problem of predicting whether a student gets admitted into a university.

LDA is a generative model for classification. Given a training dataset of positive and negative features (x, y) with $y \in \{0, 1\}$, LDA models the data x as generated from class-conditional Gaussians: $P(x, y) = P(x|y)P(y)$, where $P(y = 1) = \pi$ and $P(x|y = j) = N(x; \mu^j, \Sigma)$. Here means μ^j are class-dependent but the covariance matrix Σ is class-independent (the same for all classes).

A novel feature x is classified as a positive if $P(y = 1|x) > P(y = 0|x)$, which is equivalent to $a(x) > 0$, where the linear classifier $a(x) = w^T x + w_0$ has weights given by $w = \Sigma^{-1}(\mu^1 - \mu^0)$.

In practice, and in this assignment, we use $a(x) > \text{some threshold}$, or equivalently, $w^T x > T$ for some constant T . The specific threshold can be determined on a validation dataset or via cross-validation.

LDA and logistic regression can be expressed in the same form

$$P(y = 1|x) = \frac{1}{1 + e^{-\theta^T x}}.$$

However, they generally produce different solutions for the parameter theta.

1.1 Derivation of LDA [20 pts]

Show that the log-odds decision function $a(x)$ for LDA

$$a(x) = \ln \frac{p(x|y=1)p(y=1)}{p(x|y=0)p(y=0)}$$

is linear in x , that is, we can express $a(x) = w^T x + w_0$ for some w, w_0 . Show all your steps.

$$a(x) = \ln \frac{p(x|y=1)p(y=1)}{p(x|y=0)p(y=0)} = \ln(p(x|y=1)p(y=1)) - \ln(p(x|y=0)p(y=0))$$

$$\log(AB) = \log(A) + \log(B), \text{ so}$$

$$\ln(p(x|y=1)p(y=1)) - \ln(p(x|y=0)p(y=0)) = \ln p(x|y=1) + \ln p(y=1) - \ln p(x|y=0) - \ln p(y=0)$$

$$\ln p(x|y=1) + \ln p(y=1) - \ln p(x|y=0) - \ln p(y=0) = \ln p(x|y=1) - \ln p(x|y=0) + \ln p(y=1) - \ln p(y=0)$$

$p(x|y=1)$ is a multivariate Normal distribution, so

$$p(x|y=1) = \frac{1}{(2\pi)^{\frac{p}{2}} |\Sigma|^{-\frac{1}{2}}} e^{-\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)},$$

$$\ln p(x|y=1) = -\frac{p}{2} \ln(2\pi) - \frac{1}{2} \ln(|\Sigma|) - \frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)$$

And $p(y=1) = \pi$. This makes $p(y=0) = 1 - \pi$. Plug back in to the equation

$$\ln p(x|y=1) - \ln p(x|y=0) + \ln p(y=1) - \ln p(y=0)$$

$$\ln p(x|y=1) - \ln p(x|y=0) + \ln p(y=1) - \ln p(y=0) = -\frac{p}{2} \ln(2\pi) - \frac{1}{2} \ln(|\Sigma|) -$$

Simply this leads to

$$= -\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1) + \frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0) + \ln \pi - \ln(1-\pi)$$

expand each terms and Plug back in

$$\begin{aligned}
 (x - \mu_1)^T \Sigma^{-1} (x - \mu_1) &= x^T \Sigma^{-1} x - 2\mu_1^T \Sigma^{-1} x + \mu_1^T \Sigma^{-1} \mu_1 \\
 (x - \mu_0)^T \Sigma^{-1} (x - \mu_0) &= x^T \Sigma^{-1} x - 2\mu_0^T \Sigma^{-1} x + \mu_0^T \Sigma^{-1} \mu_0 \\
 &= -\frac{1}{2}(x^T \Sigma^{-1} x) + \mu_1^T \Sigma^{-1} x - \frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}(x^T \Sigma^{-1} x) - \mu_0^T \Sigma^{-1} x + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \\
 &= \mu_1^T \Sigma^{-1} x - \frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 - \mu_0^T \Sigma^{-1} x + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \ln \pi - \ln(1 - \pi)
 \end{aligned}$$

Rewrite this respect to x.

$$= (\mu_1^T - \mu_0^T) \Sigma^{-1} x - \frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \ln \pi - \ln(1 - \pi)$$

So in $w^T x + w_0$ form, $w^T = (\mu_1^T - \mu_0^T) \Sigma^{-1}$,
 $w_0 = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \ln \pi - \ln(1 - \pi)$

Implementation

In this part, you can assume the prior probabilities for the two classes are the same (although the number of the positive and negative samples in the training data is not the same), and that the threshold T is zero (you do not need to find w_0). Later you will explore how the different prior probabilities shift the decision boundary.

1.2 Loading the data

First, lets load and plot the data.

```

In [3]: %matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

datafile = 'data/ex2data1.txt'

cols = np.loadtxt(datafile, delimiter=',', usecols=(0, 1, 2), unpack=True)
##Form the usual "X" matrix and "y" vector
X = np.transpose(np.array(cols[:-1]))
y = np.transpose(np.array(cols[-1:]))
m = y.size # number of training examples
##Insert the usual column of 1's into the "X" matrix
X = np.insert(X, 0, 1, axis=1)

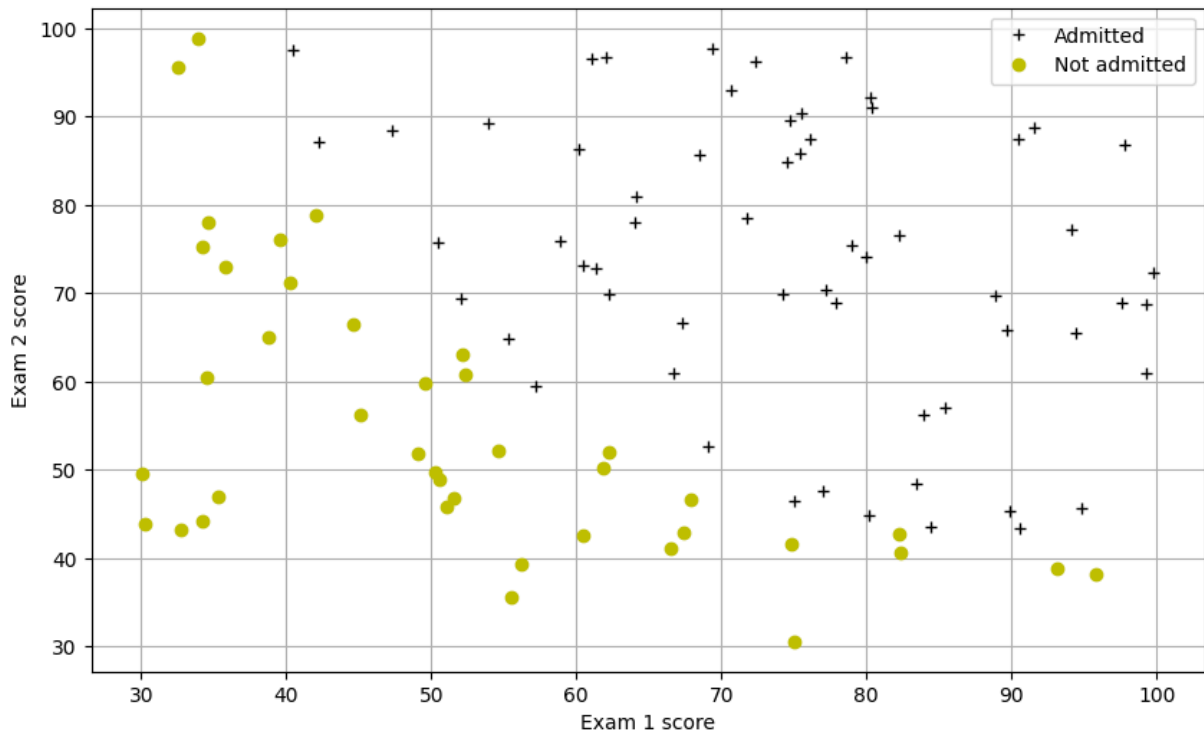
#Divide the sample into two: ones with positive classification, one with nul
pos = np.array([X[i] for i in range(X.shape[0]) if y[i] == 1])
neg = np.array([X[i] for i in range(X.shape[0]) if y[i] == 0])

def plotData():
    plt.figure(figsize=(10, 6))

```

```
plt.plot(pos[:, 1], pos[:, 2], 'k+', label='Admitted')
plt.plot(neg[:, 1], neg[:, 2], 'yo', label='Not admitted')
plt.xlabel('Exam 1 score')
plt.ylabel('Exam 2 score')
plt.legend()
plt.grid(True)
```

plotData()



Implement the LDA classifier by completing the code below.

1.3 Centering the data [5 pts]

As an implementation detail, you should first center the positive and negative data separately, so that each set has a mean equal to $[0, 0]$, before computing the covariance, as this tends to give a more accurate estimate of the covariance.

```
In [4]: ##### IMPLEMENT THIS #####
pos_mean = np.mean(pos[:, 1:], axis=0)
neg_mean = np.mean(neg[:, 1:], axis=0)

pos_data = pos[:, 1:] - pos_mean
neg_data = neg[:, 1:] - neg_mean
```

```
In [5]: print(np.mean(pos_data, axis=0))
print(np.mean(neg_data, axis=0))
```

```
[-8.05281767e-15  1.33818882e-14]
[1.54543045e-14  9.76996262e-15]
```

1.4 Computing parameters and predictions [10 pts]

Implement the LDA algorithm here in vectorized form (avoid loops). First, compute the covariance matrix, then the classifier's weights, then use the classifier to make predictions on the training data. Note, you should center the whole training data set before applying the classifier. Namely, subtract the mean value of the two classes' means ($1/2 * (\text{pos_mean} + \text{neg_mean})$), which is on the separating plane when their prior probabilities are the same and becomes the 'center' of the data.

```
In [24]: X_total_mean = 1/2 * (pos_mean + neg_mean) # shape: (2,)
X_centered = X[:, 1:] - X_total_mean # SHAPE: (100,2)
pos_mean_centered = (pos_mean - X_total_mean).reshape(2, 1) # SHAPE: (2,1)
neg_mean_centered = (neg_mean - X_total_mean).reshape(2, 1) # SHAPE: (2,1)

cov_all = np.cov(X_centered.T) # SHAPE: (2,2)
inverse_cov = np.linalg.inv(cov_all) # SHAPE: (2,2)

w = (pos_mean_centered.T - neg_mean_centered.T) @ inverse_cov # SHAPE: (2,
w0 = -1/2 * ( pos_mean_centered.T @ inverse_cov @ pos_mean_centered) + 1/2 *
y_lda = np.where((X_centered @ w.T + w0) < 0, 0, 1 ) # SHAPE: (100,)

print("pos_mean_centered", pos_mean_centered.shape)
print("neg_mean_centered", neg_mean_centered.shape)
print("inverse_cov", inverse_cov.shape)
print("w", w.shape)
print("w0", w0.shape)
print("y_lda", y_lda.shape)

pos_mean_centered (2, 1)
neg_mean_centered (2, 1)
inverse_cov (2, 2)
w (1, 2)
w0 (1, 1)
y_lda (100, 1)
```

```
In [36]: print(w0)

[[2.22044605e-16]]
```

1.5 Training accuracy [5 pts]

Complete the code to compute the training set accuracy, which should be around 89%.

```
In [25]: ##### IMPLEMENT THIS #####
accuracy = np.mean(y_lda == y) * 100
print(accuracy)
```

89.0

1.6 Effect of prior [10 pts]

Try changing the prior probability of the positive class $P(y = 1) = \pi$ and show how the resulting decision boundary changes by plotting it. What is the effect of changing π on the boundary?

```

In [ ]: # Plot decision boundary for various biases.
# Changing the prior probability of y=1 only affects the bias, specifically
# term  $\log(p(y=1)/p(y=0))$  which is part of the bias term. The effect is to move
# the decision boundary away from the more prevalent class. So if we change
# to make class 1 ("+") more prevalent, the bias should increase.

# center the data, split into classes (pos "+" and neg "o")
X_c = X[:,1:] - np.kron(np.ones((len(X), 1)), (1/2*(pos_mean + neg_mean)))
pos_c = np.array([X_c[i] for i in range(X_c.shape[0]) if y[i] == 1])
neg_c = np.array([X_c[i] for i in range(X_c.shape[0]) if y[i] == 0])

# make a new figure and plot points
plt.figure(figsize=(10, 6))
plt.plot(pos_c[:, 0], pos_c[:, 1], 'k+', label='Admitted')
plt.plot(neg_c[:, 0], neg_c[:, 1], 'yo', label='Not admitted')
plt.xlabel('Exam 1 score')
plt.ylabel('Exam 2 score')
plt.legend()
plt.grid(True)

##### IMPLEMENT THIS #####
# Show how the decision boundary changes with prior.
# Plotting the decision boundary: two points, draw a line between them
# Decision boundary occurs when  $h(x) = 0$ , ie when
#  $w_0 * x_1 + w_1 * x_2 + \text{bias} = 0$ 
# So the line  $y=mx+b$  is given by  $x_2 = (-1/w_1)(\text{bias} + w_0 * x_1)$ 
pi = len(pos_c) / len(X_c)
print("pi", pi)
log_odds = np.log(pi / (1 - pi))
print(f"π: {pi:.2f}, log odds: {log_odds:.2f}")
w0_, w1_ = w.flatten()

x1_vals = np.linspace(X_c[:, 0].min(), X_c[:, 0].max(), 100)

bias_0 = 0
bias_2 = 2

# decision boundary:  $x_2 = -(\text{bias} + w_0 * x_1) / w_1$ 
x2_vals_bias0 = -(bias_0 + w0_ * x1_vals) / w1_
x2_vals_bias2 = -(bias_2 + w0_ * x1_vals) / w1_

# bias = 0
plt.plot(x1_vals, x2_vals_bias0, color='cornflowerblue', linewidth=8, alpha=0.7)

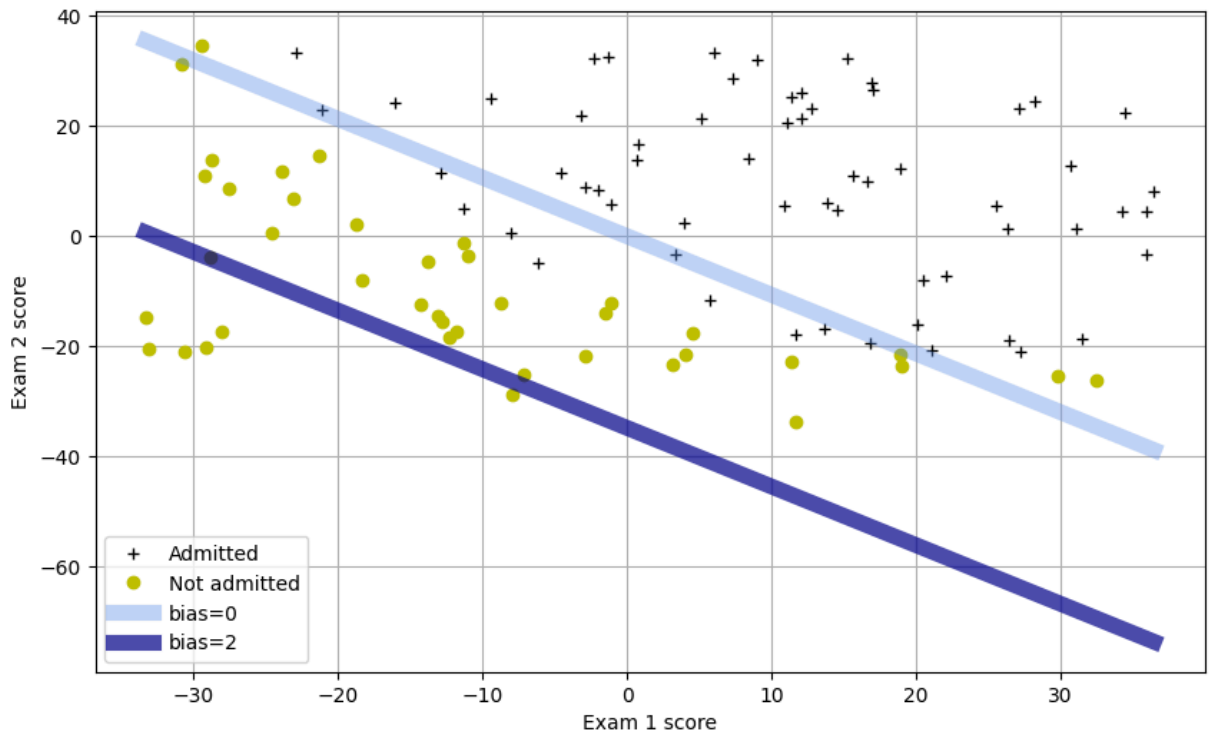
# bias = 2
plt.plot(x1_vals, x2_vals_bias2, color='darkblue', linewidth=8, alpha=0.7, 1

plt.legend()
plt.show()

```

pi 0.6

π: 0.60, log odds: 0.41



2 Conjugate Prior [25 pts]

In this problem we will estimate the parameter θ of the coin flipping problem using the Bayesian approach. In the Bayesian approach, θ is considered a random variable sampled from some prior distribution $p(\theta)$. We represent the i th coin flip by a Bernoulli random variable $x \in \{0, 1\}$, where $x_i = 1$ if the i th flip lands on heads. We assume that the observed x_i 's are conditionally independent given θ . This means that the joint distribution of n coin flips and θ can be factorized as

$$p(x_1, \dots, x_n, \theta) = p(\theta) \prod_{i=1}^n p(x_i | \theta)$$

Suppose that our prior distribution on θ is $Beta(h, t)$, for some $h, t > 0$. That is,

$$p(\theta) \propto \theta^{h-1} (1 - \theta)^{t-1}$$

See Bishop Ch 2.1.1 or https://en.wikipedia.org/wiki/Beta_distribution for more details on this distribution. There is a nice online graphing tool here <http://eurekastatistics.com/beta-distribution-pdf-grapher/>.

The Beta distribution is a conjugate to the Bernoulli distribution since the prior $p(\theta)$ and posterior $p(\theta | x_1, \dots, x_n)$ are in the same family (the Beta family).

2.1 Posterior [10 pts]

Suppose our dataset of flips has H heads and T tails. Show that the posterior distribution for θ is $Beta(h + H, t + T)$, i.e. show that

$$p(\theta|x_1, \dots, x_n) \propto \theta^{h-1+H}(1-\theta)^{t-1+T}$$

I have the fact that prior distribution on θ is $Beta(h, t)$ for some $h, t > 0$. That is,

$$p(\theta) \propto \theta^{h-1} \times (1-\theta)^{t-1}$$

Given that when the dataset of flips has H heads and T tails, the likelihood distribution, $p(x_1, \dots, x_n|\theta)$ follows a Binomial distribution,

$$Binomial(H + T, \theta) = \binom{H+T}{H} \theta^H (1-\theta)^T \propto \theta^H (1-\theta)^T.$$

So the Posterior,

$$P(\theta|x_1, \dots, x_n) = p(\theta) \times p(x_1, \dots, x_n|\theta) \propto \theta^{h-1} \times (1-\theta)^{t-1} \times \theta^H (1-\theta)^T = \theta^{h-1+H} (1-\theta)^{t-1+T}$$

2.2 Parameter estimates [10 pts]

Give the expressions for three estimates of θ : the Maximum Likelihood (θ_{ML}), the Maximum a Posteriori (θ_{MAP}) or the mode of the posterior over θ , and the mean of the posterior (θ_{MP}).

Hint: You may use the fact that a Beta(h, t) distribution has mean $h/(h+t)$ and has mode $(h-1)/(h+t-2)$ for $h, t > 1$.

1.) Maximum Likelihood(θ_{ML})

I have the likelihood function θ_{ML} is $p(x_1, \dots, x_n|\theta) = \theta^H \times (1-\theta)^T$. In order to get the maximum of this function, I use the log likelihood function.

$$\log(p(x|\theta)) = H \log \theta + T \log(1-\theta)$$

Find the θ that makes derivative of this log likelihood function 0.

$$\frac{d}{d\theta} (H \log \theta + T \log(1-\theta)) = 0$$

$$\frac{H}{\theta} + \frac{-T}{1-\theta} = 0$$

$$\frac{H}{\theta} = \frac{T}{1-\theta}$$

$$H * (1-\theta) = T * \theta$$

$$(H - H\theta) = T\theta$$

$$H = T\theta + H\theta$$

$$H = (T + H)\theta$$

$$\theta = \frac{H}{T + H}$$

2.) Maximum A Posteriori(θ_{MAP}) (Mode of the Posterior over θ)

Maximum A Posteriori(θ_{MAP}) is the Mode of the Posterior over θ . I have the posterior formula, $\theta^{(h-1+H)}(1 - \theta)^{t-1+T}$. The hint says mode of the beta distribution, so I can apply to this posterior formula.

$$\text{Mode of } \text{Beta}(h + H, t + T) = \frac{h + H - 1}{h + H + t + T - 2}$$

3.) Mean of the Posterior (θ_{MP})

Using the same mean formula to this posterior.

$$\text{Mean of } \text{Beta}(h + H, t + T) = \frac{h + H}{h + H + t + T}$$

2.3 Example [5 pts]

Suppose you are modeling some event with a binary outcome (such as a user clicking on an ad after viewing it). You know that typical fractions of positive events observed in data are between 0.01 and 0.02. What are reasonable hyperparameters of the Beta prior to use, and why? Include a plot of the resulting prior PDF.

A reasonable choice for the Beta prior is to set the hyperparameters h and t so that the prior mean reflects the expected value of θ .

The mean of a Beta(h, t) distribution is:

$$\text{mean} = \frac{h}{h+t}$$

If we expect θ to be between 0.01 and 0.02, we should choose h and t such that the prior mean lies within this range.

For example, setting $h = 2$ and $t = 98$ results in a mean of 0.02. The total $h+t = 100$ provides a moderately informative prior, meaning that the prior reflects our belief about θ while still allowing the observed data to influence the posterior significantly.

Thus, the Beta(2, 98) prior is appropriate because: • The prior mean aligns with the expected value of θ . • The prior distribution is not too sharp or overly confident, maintaining flexibility for updating based on new data.

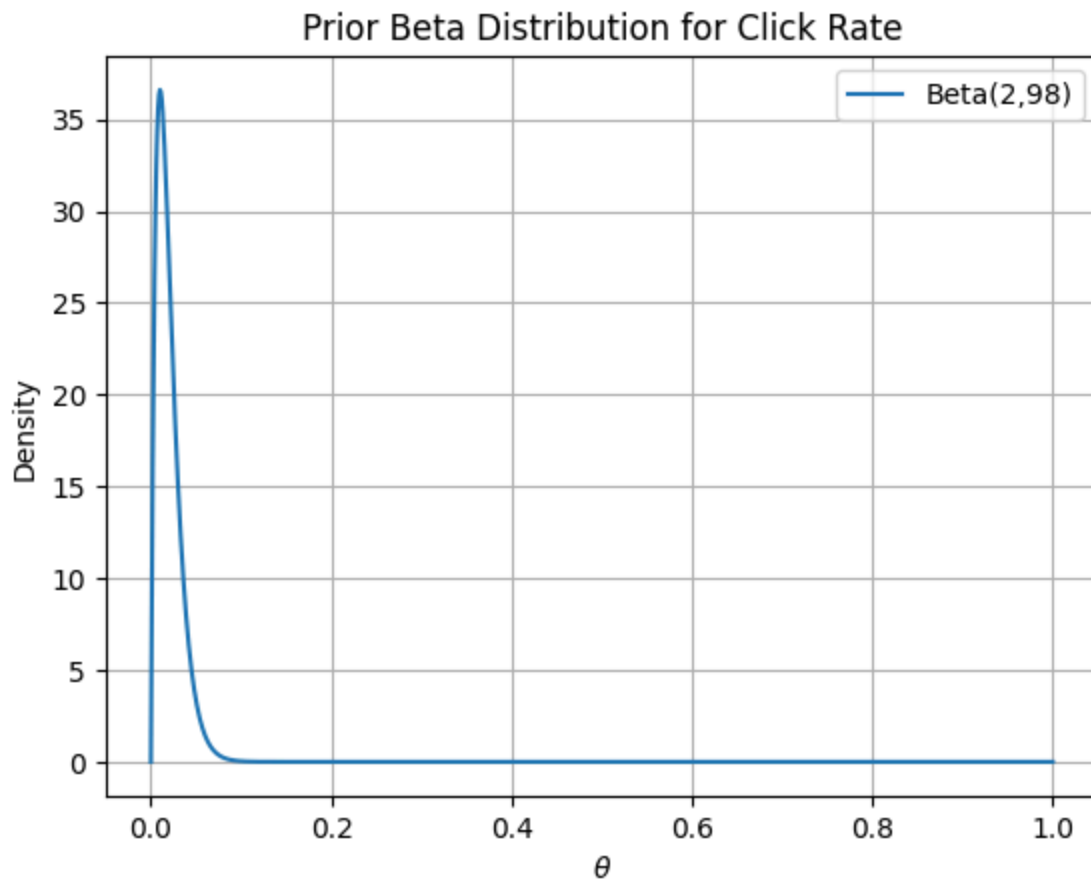
Here is a plot of the resulting Beta(2, 98) prior distribution:

```
In [115... import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta

alpha = 2
beta_param = 98

theta = np.linspace(0, 1, 1000)
pdf = beta.pdf(theta, alpha, beta_param)

plt.plot(theta, pdf, label=f'Beta({alpha},{beta_param})')
plt.xlabel(r'$\theta$')
plt.ylabel('Density')
plt.title('Prior Beta Distribution for Click Rate')
plt.legend()
plt.grid()
plt.show()
```



3 Bayesian Linear Regression [25 pts + 20 pts bonus]

In this exercise, we implement Bayesian regression with linear basis function models. Recall that a linear regression model $y(\mathbf{x}, \mathbf{w})$ can be defined more generally as

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$$

where ϕ_j are basis functions and M is the total number of parameters w_j including the bias term w_0 . The target variable t of an observation $\mathbf{x}$ is given by a deterministic function $y(\mathbf{x}, \mathbf{w})$ plus additive random noise ϵ .

$$t = y(\mathbf{x}, \mathbf{w}) + \epsilon$$

We assume that the noise is normally distributed i.e. follows a Gaussian distribution with zero mean and precision (inverse variance) β . The corresponding probabilistic model i.e. the conditional distribution of t given $\mathbf{x}$ can therefore be written as

$$p(t|\mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \beta^{-1}) = \sqrt{\frac{\beta}{2\pi}} \exp\left(-\frac{\beta}{2}(t - y(\mathbf{x}, \mathbf{w}))^2\right)$$

where the mean of this distribution is the regression function $y(\mathbf{x}, \mathbf{w})$.

3.1 Posterior and posterior predictive distribution [10pts]

For a Bayesian treatment of linear regression, we need a prior probability distribution over the model parameters $\mathbf{w}$ with zero mean:

$$p(\mathbf{w}|\alpha) = \mathcal{N}(\mathbf{w}|0, \alpha^{-1}\mathbf{I})$$

where $\alpha^{-1}\mathbf{I}$ is the diagonal covariance matrix where all diagonal elements have the same variance α^{-1} .

The posterior distribution has the following mean and inverse covariance matrix:

$$m_N = \beta \mathbf{S}_N \Phi^T \mathbf{t}$$

$$\mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \Phi^T \Phi$$

Hence, the posterior distribution can be written as

$$p(\mathbf{w}|\mathbf{t}, \alpha, \beta) = \mathcal{N}(\mathbf{w}|\mathbf{m}_N, \mathbf{S}_N)$$

For making a prediction t at a new location $\mathbf{x}$ we use the posterior predictive distribution which is defined as

$$p(t|\mathbf{t}, \mathbf{w}, \alpha, \beta) = \int p(t|\mathbf{x}, \mathbf{w}, \beta) p(\mathbf{w}|\mathbf{t}, \alpha, \beta) d\mathbf{w}$$

Implement the posterior and posterior predictive distributions.

```
In [64]: import numpy as np

def posterior(Phi, t, alpha, beta, return_inverse=False):
    """Computes mean and covariance matrix of the posterior distribution."""
    S_N = np.linalg.inv(alpha * np.eye(Phi.shape[1]) + beta * Phi.T @ Phi)
    S_N_inv = np.linalg.inv(S_N)
```

```

m_N = beta * S_N @ Phi.T @ t

if return_inverse:
    return m_N, S_N, S_N_inv
else:
    return m_N, S_N

def posterior_predictive(Phi_test, m_N, S_N, beta):
    """Computes mean and variances of the posterior predictive distribution.
    y = Phi_test @ m_N
    y_var = 1 / beta + np.sum(Phi_test @ S_N * Phi_test, axis=1)

    return y, y_var

```

3.2 Generating the data [5pts]

Target values t are generated from the design matrix $\mathbf{X} \in \mathbb{R}^{N \times 1}$ with a linear function f which can also generate random noise of specified variance. Implement f below.

```

In [65]: w0 = -0.3 # line offset
         w1 = 0.5 # line slope

def f(X, noise_variance):
    '''Linear function plus noise'''
    noise_val = noise(X.shape[0], noise_variance).reshape(-1, 1)
    t = w1 * X + w0 + noise_val

    return t

def noise(size, variance):
    return np.random.normal(scale=np.sqrt(variance), size=size)

```

3.3 Basis functions [5pts]

For straight line fitting, we do not need to transform x with a basis function, which is equivalent to using an identity basis function. Other basis functions that are non-linear are necessary to model the non-linear relationship between input x and target t . Below is an example of non-linear basis functions: the polynomial. Implement the polynomial basis function. *Note: You will not use it in the rest of the exercise but your implementation should be reasonable.*

```

In [66]: def identity_basis_function(x):
         return x

def polynomial_basis_function(x, power):
    polynomial_x = np.hstack([x**p for p in range(power + 1)]) # each data c
    return polynomial_x

```

```
def expand(x, bf, bf_args=None):
    if bf_args is None:
        return np.concatenate([np.ones(x.shape), bf(x)], axis=1)
    else:
        return np.concatenate([np.ones(x.shape)] + [bf(x, bf_arg) for bf_arg
```

3.3 Straight line fitting [5pts]

For straight line fitting, we use a linear regression model of the form

$y(\mathbf{w}, \mathbf{x}) = w_0 + w_1x$ and perform Bayesian inference for model parameters $\mathbf{w}$.

Predictions are made with the posterior predictive distribution. Since this model has only two parameters, w_0 and w_1 , we can visualize the posterior density in 2D which is done in the first column of the following output. Rows use an increasing number of training data from a training dataset.

Question: Run the code below. How does the dataset size affect the posterior density in the first column of the plots?

Answer: As the number of the data increases, the density of the posterior increases and have a smaller, accurate density.

```
In [67]: from scipy import stats
import matplotlib.pyplot as plt

def plot_data(x, t):
    plt.scatter(x, t, marker='o', c="k", s=20)

def plot_truth(x, y, label='Truth'):
    plt.plot(x, y, 'k--', label=label)

def plot_predictive(x, y, std, y_label='Prediction', std_label='Uncertainty'):
    y = y.ravel()
    std = std.ravel()

    plt.plot(x, y, label=y_label)
    plt.fill_between(x.ravel(), y + std, y - std, alpha = 0.5, label=std_label)

    if plot_xy_labels:
        plt.xlabel('x')
        plt.ylabel('y')

def plot_posterior_samples(x, ys, plot_xy_labels=True):
    plt.plot(x, ys[:, 0], 'r-', alpha=0.5, label='Post. samples')
    for i in range(1, ys.shape[1]):
        plt.plot(x, ys[:, i], 'r-', alpha=0.5)

    if plot_xy_labels:
        plt.xlabel('x')
        plt.ylabel('y')
```

```

def plot_posterior(mean, cov, w0, w1):
    resolution = 100

    grid_x = grid_y = np.linspace(-1, 1, resolution)
    grid_flat = np.dstack(np.meshgrid(grid_x, grid_y)).reshape(-1, 2)

    densities = stats.multivariate_normal.pdf(grid_flat, mean=mean.ravel(),
    plt.imshow(densities, origin='lower', extent=(-1, 1, -1, 1))
    plt.scatter(w0, w1, marker='x', c="r", s=20, label='Truth')

    plt.xlabel('w0')
    plt.ylabel('w1')

def print_comparison(title, a, b, a_prefix='np', b_prefix='br'):
    print(title)
    print('-' * len(title))
    print(f'{a_prefix}:', a)
    print(f'{b_prefix}:', b)
    print()

```

```

In [68]: import matplotlib.pyplot as plt
%matplotlib inline

# fix random seed so we always get the same data
np.random.seed(5)

# Training dataset sizes
N_list = [1, 3, 20]

beta = 25.0
alpha = 2.0

# Training observations in [-1, 1)
X = np.random.rand(N_list[-1], 1) * 2 - 1

# Training target values
t = f(X, noise_variance=1/beta)

# Test observations
X_test = np.linspace(-1, 1, 100).reshape(-1, 1)

# Function values without noise
y_true = f(X_test, noise_variance=0)

# Design matrix of test observations
Phi_test = expand(X_test, identity_basis_function)

plt.figure(figsize=(15, 10))
plt.subplots_adjust(hspace=0.4)

for i, N in enumerate(N_list):
    X_N = X[:N]
    t_N = t[:N]

```

```
# Design matrix of training observations
Phi_N = expand(X_N, identity_basis_function)

# Mean and covariance matrix of posterior
m_N, S_N = posterior(Phi_N, t_N, alpha, beta)

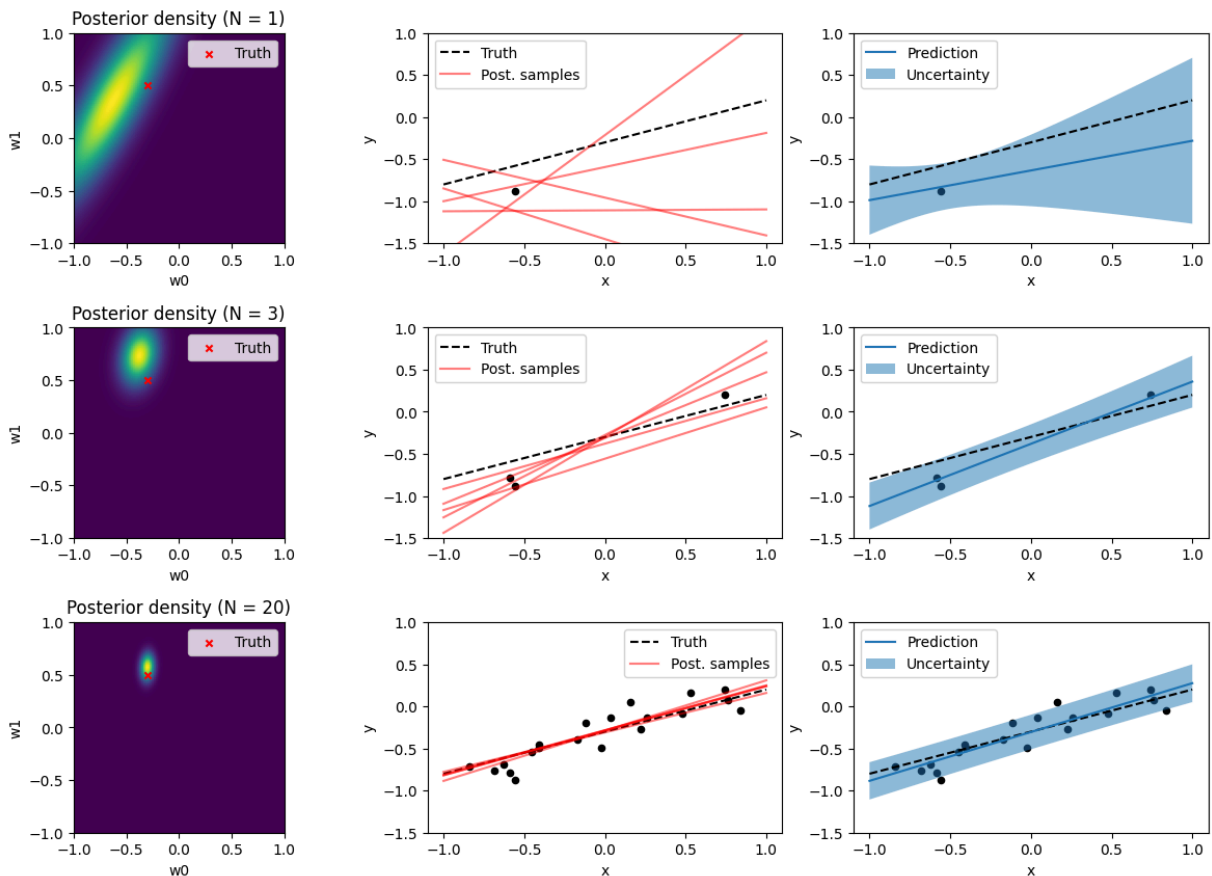
# Mean and variances of posterior predictive
y, y_var = posterior_predictive(Phi_test, m_N, S_N, beta)

# Draw 5 random weight samples from posterior and compute y values
w_samples = np.random.multivariate_normal(m_N.ravel(), S_N, 5).T
y_samples = Phi_test.dot(w_samples)

plt.subplot(len(N_list), 3, i * 3 + 1)
plot_posterior(m_N, S_N, w0, w1)
plt.title(f'Posterior density (N = {N})')
plt.legend()

plt.subplot(len(N_list), 3, i * 3 + 2)
plot_data(X_N, t_N)
plot_truth(X_test, y_true)
plot_posterior_samples(X_test, y_samples)
plt.ylim(-1.5, 1.0)
plt.legend()

plt.subplot(len(N_list), 3, i * 3 + 3)
plot_data(X_N, t_N)
plot_truth(X_test, y_true, label=None)
plot_predictive(X_test, y, np.sqrt(y_var))
plt.ylim(-1.5, 1.0)
plt.legend()
```



3.4 Bonus [20pts] (optional)

We now fit a Gaussian basis function model to a noisy sinusoidal dataset for which we provide a sinusoidal function g with noise variance. Implement the gaussian basis function below.

The model uses 7 Gaussian basis functions with mean values equally distributed over $[0,1]$ each having a standard deviation of 0.1. We then infer the model parameters $\mathbf{w}$ using the posterior predictive distribution.

How many parameters w_i are there? Run Bayesian inference and generate the plots. Do they follow the same trend as the ones generated by Bayesian linear regression?

Answer:

```
In [69]: def g(X, noise_variance):
    '''Sinusoidal function plus noise'''
    return 0.5 + np.sin(2 * np.pi * X) + noise(X.shape, noise_variance)

    def gaussian_basis_function(x, mu, sigma=0.1):
        return np.exp(-0.5 * ((x - mu) / sigma) ** 2)
```

```
In [70]: import matplotlib.pyplot as plt
    %matplotlib inline
    from scipy.optimize import curve_fit
```

```

# fix random seed so we always get the same data
np.random.seed(5)

N_list = [3, 8, 20]

beta = 25.0
alpha = 2.0

# Training observations in [-1, 1)
X = np.random.rand(N_list[-1], 1)

# Training target values
t = g(X, noise_variance=1/beta)

# Test observations
X_test = np.linspace(0, 1, 100).reshape(-1, 1)

# Function values without noise
y_true = g(X_test, noise_variance=0)

# Design matrix of test observations
Phi_test = expand(X_test, bf=gaussian_basis_function, bf_args=np.linspace(0, 1

plt.figure(figsize=(10, 10))
plt.subplots_adjust(hspace=0.4)

for i, N in enumerate(N_list):
    X_N = X[:N]
    t_N = t[:N]

    # Design matrix of training observations
    Phi_N = expand(X_N, bf=gaussian_basis_function, bf_args=np.linspace(0, 1

    # Mean and covariance matrix of posterior
    m_N, S_N = posterior(Phi_N, t_N, alpha, beta)

    # Mean and variances of posterior predictive
    y, y_var = posterior_predictive(Phi_test, m_N, S_N, beta)

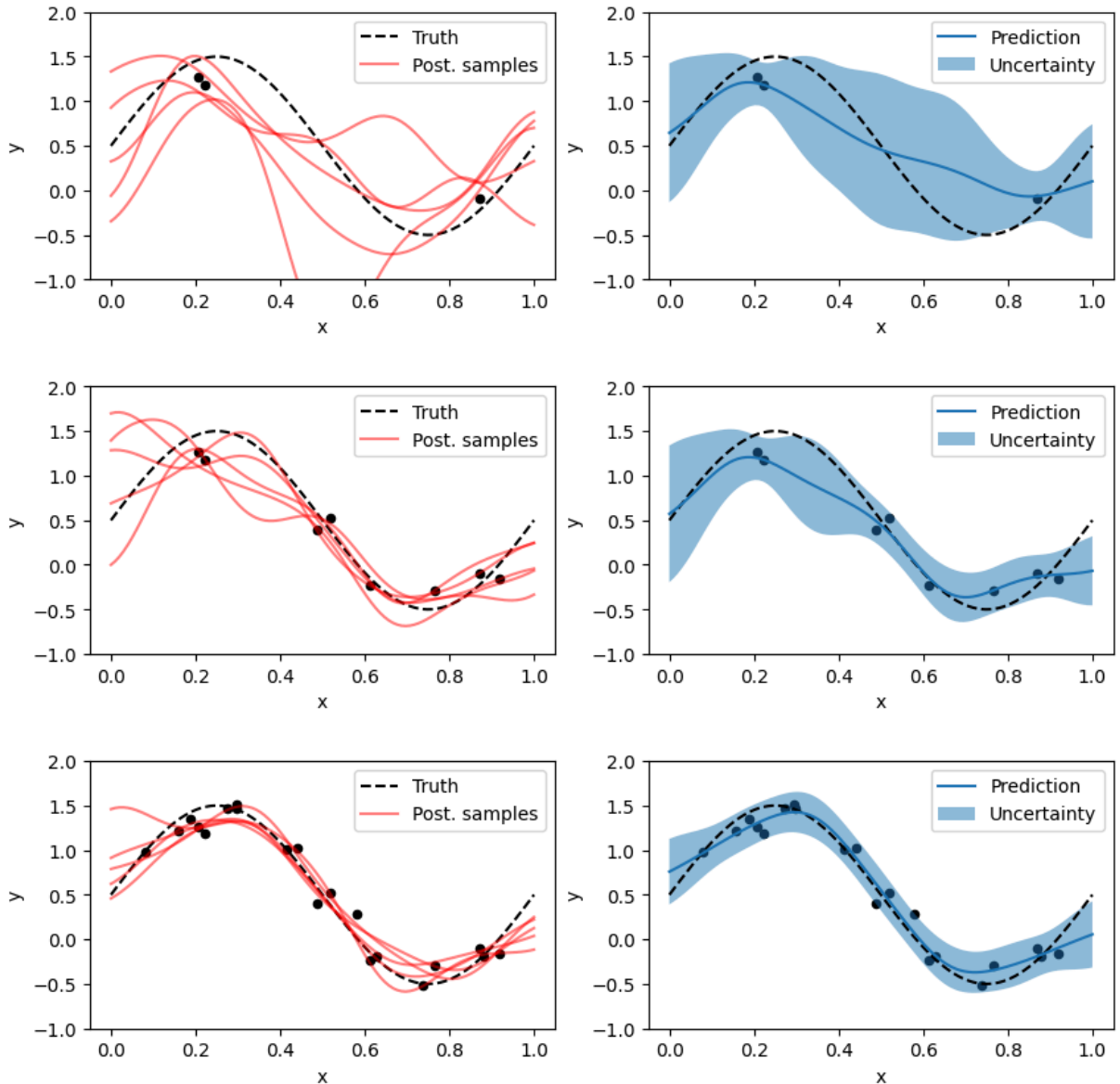
    # Draw 5 random weight samples from posterior and compute y values
    w_samples = np.random.multivariate_normal(m_N.ravel(), S_N, 5).T
    y_samples = Phi_test.dot(w_samples)

    plt.subplot(len(N_list), 2, i * 2 + 1)
    plot_data(X_N, t_N)
    plot_truth(X_test, y_true)
    plot_posterior_samples(X_test, y_samples)
    plt.ylim(-1.0, 2.0)
    plt.legend()

    plt.subplot(len(N_list), 2, i * 2 + 2)
    plot_data(X_N, t_N)
    plot_truth(X_test, y_true, label=None)
    plot_predictive(X_test, y, np.sqrt(y_var))

```

```
plt.ylim(-1.0, 2.0)
plt.legend()
```



4 Reinforcement Learning [25 pts]

In this problem, you will implement the Q-Learning Algorithm to solve the "Frozen Lake" problem. We will use OpenAI's gym package to develop our solution in Python.

OpenAI Gym

Gym is a standard API for reinforcement learning, and a diverse collection of reference environments. You can read the basic setup instructions for Gym [here](#).

Frozen Lake

We're going to be using the [FrozenLake](#) environment. You can read through the code for this environment by following the Github link.



Winter is quickly approaching, and the agent has to worry about navigating frozen lakes. It's only early November, so the lakes haven't completely frozen and if you make the wrong step you may fall through! The agent will need to learn how to get to the destination when stuck on the ice, without falling in. The lake we're going to consider is a square lake with spots to step on in the shape of a grid.

The surface is described using a 4x4 grid of "states":

```
S F F F
F H F H
F F F H
H F F G
```

The corresponding state ids are

```
0  1  2  3
4  5  6  7
8  9 10 11
12 13 14 15
```

Each spot can have one of four states:

- S: starting point.
- G: goal point.
- F: frozen spot, where it's safe to walk.
- H: hole in the ice, where it's not safe to walk.

The action space (the set of possible actions the agent can take) is discrete with 4 actions available 🎮:

- 0: GO LEFT

- 1: GO DOWN
- 2: GO RIGHT
- 3: GO UP

Reward function 💰 :

- Reach goal: +1
- Reach hole: 0
- Reach frozen: 0

Although we can see the path we need to walk, the agent does not. We're going to train an agent to discover this via reinforcement learning. However, walking on ice isn't so easy! Sometimes you slip and aren't able to take the step you intended. In our environment, this is controlled by setting `is_slippery: True/False`. If `True`, each step will move in intended direction with probability of 1/3, else will move in either perpendicular direction with equal probability of 1/3 in each directions.

The episode ends when the agent reaches the goal or falls in a hole. The agent receives a reward of 1 upon reaching the goal, and zero otherwise.

4.0 Install dependencies

```
In [34]: !pip install gym==0.25.2
!pip install pygame
!pip install numpy
```

```
Requirement already satisfied: gym==0.25.2 in /opt/homebrew/lib/python3.10/site-packages (0.25.2)
Requirement already satisfied: numpy>=1.18.0 in /opt/homebrew/lib/python3.10/site-packages (from gym==0.25.2) (1.26.4)
Requirement already satisfied: cloudpickle>=1.2.0 in /opt/homebrew/lib/python3.10/site-packages (from gym==0.25.2) (3.1.1)
Requirement already satisfied: gym_notices>=0.0.4 in /opt/homebrew/lib/python3.10/site-packages (from gym==0.25.2) (0.0.8)
Requirement already satisfied: pygame in /opt/homebrew/lib/python3.10/site-packages (2.6.1)
Requirement already satisfied: numpy in /opt/homebrew/lib/python3.10/site-packages (1.26.4)
```

4.1. Walking on the Frozen Lake [5 points]

The script below sets up the Frozen Lake environment and takes 10 walks through it, consisting of a maximum 10 randomly sampled actions during each walk. After each step, it prints the current state id, and prints out the reward and whether the walk is "done", i.e. in the terminal state, because the agent fell into a hole (stop if it is).

The environment behaves in a stochastic way, i.e. taking an action does not deterministically transition the agent into the next state. Instead the agent ends up in

one of several states with some probability. The analogy here is that when you try to walk right on ice, you may slip and end up walking forward, etc.

Run the script and make sure you understand the behavior of the agent. Is it easy for the agent to reach the goal while taking random actions? Read the code to understand what it's doing (there is nothing to implement for this part).

```
In [58]: import numpy as np
import gym
```

```
In [59]: #env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=False)
env = gym.make("FrozenLake-v1", new_step_api=True).unwrapped
np.random.seed(42)
actions = ["LEFT", "DOWN", "RIGHT", "UP"]

# Do a random walk
for i in range(2):
    env.reset()
    print("Walk %d" % i)
    for _ in range(10):
        # env.render() # note, this does not work in Colab
        action = env.action_space.sample()
        next_state, reward, done, info, prob = env.step(action) # take a random
        print("Action %s \t State %d Reward %d done? %d" % (actions[action], next_state, reward, done))
    env.close()
```

```
Walk 0
Action UP      State 0 Reward 0 done? 0
Action RIGHT   State 0 Reward 0 done? 0
Action RIGHT   State 4 Reward 0 done? 0
Action RIGHT   State 5 Reward 0 done? 1
Action UP      State 5 Reward 0 done? 1
Action DOWN    State 5 Reward 0 done? 1
Action LEFT    State 5 Reward 0 done? 1
Action DOWN    State 5 Reward 0 done? 1
Action LEFT    State 5 Reward 0 done? 1
Action LEFT    State 5 Reward 0 done? 1
Walk 1
Action DOWN    State 4 Reward 0 done? 0
Action RIGHT   State 5 Reward 0 done? 1
Action DOWN    State 5 Reward 0 done? 1
Action DOWN    State 5 Reward 0 done? 1
Action UP      State 5 Reward 0 done? 1
Action DOWN    State 5 Reward 0 done? 1
Action UP      State 5 Reward 0 done? 1
Action LEFT    State 5 Reward 0 done? 1
Action LEFT    State 5 Reward 0 done? 1
Action UP      State 5 Reward 0 done? 1
```

4.2 Q-Learning Implementation [20 points]

Below we have provided most of the code to solve the FrozenLake problem, including setting up the Gym environment, calling the Q-learning function, and printing out the

returned Q-table.

Your task is to understand the provided code and fill in the missing part. Run the code, provide the final Q-table of FrozenLake for **each** `<num_episode, init_method>` tried, and analyze what you observe.

Notes

We use the ϵ -greedy algorithm (solving Exploration-Exploitation Dilemma) to generate actions for each state. The code tries `num_episodes` with different values, e.g. `num_episodes=500, 1000, 5000, 10000, 50000`. It also tries different initializations for the Q-table, i.e. Random Initialization and Zero Initialization.

Please do not change any default value of environment setting, such as `is_slippery`.

To test the learned Q-table, we can compute the win rate (the probability of agents getting to goal following the Q-table). Note that in this case, we should use `argmax` instead of epsilon greedy to choose the action at the current state. Win Rate is a good way to test whether you write your algorithm correctly.

Just for your reference, we tested our implementation several times with 1000 trials after 50000-episode training using zero initialization, and were able to achieve a win rate of 0.5 or higher on several of the trials.

```
In [60]: # whether to initialize Q-table randomly
RAND = False

""" Performs Q-learning for the given environment.
Initialize Q to all zeros or randomly.
:param env: Unwrapped OpenAI gym environment.
:param alpha: Learning rate parameter.
:param gamma: Decay rate parameter.
:param num_episodes: Number of episodes to use for learning.
"""

def q_learning(env, alpha=0.5, gamma=0.95, epsilon=0.1, num_episodes=500):
    np.random.seed(42)
    # initialize Q
    state_num = env.observation_space.n
    action_num = env.action_space.n
    Q = np.zeros((state_num, action_num))

    if RAND:
        Q = np.random.rand(state_num, action_num)
        terminal_state = [5, 7, 11, 12, 15]
        Q[terminal_state, :] = 0

    for episode in range(num_episodes):
        current_state = env.reset() # reset() returns observation, reward, te
        done = False
        while not done:
            if np.random.rand() < 1- epsilon:
```

```

        action = np.argmax(Q[current_state,:])
    else:
        action = np.random.randint(0,action_num)

    # IMPLEMENT THIS (see lecture notes for pseudocode):
    # - carry out selected action
    # - observe reward and new state
    # - update the Q table
    # - update current state to new state

    # carry out selected action
    next_state, reward, done, info, prob = env.step(action)

    # update Q table
    Q[current_state, action] = Q[current_state, action] + alpha * (

    # update current state
    current_state = next_state

    return Q

def winrate(Q, env, num_episodes):
    num_win = 0.

    for episode in range(num_episodes):
        current_state = env.reset()
        done = False
        while not done:
            action = np.argmax(Q[current_state, :])
            next_state, reward, done, info, prob = env.step(action)
            num_win += reward
            current_state = next_state

    return num_win/num_episodes

# MAIN function that runs Q-learning for Frozen Lake
def main():
    env = gym.make("FrozenLake-v1", new_step_api=True).unwrapped
    np.random.seed(42)

    print("output of Q-learning")
    num_episodes = [ 500, 1000, 5000, 10000, 50000]
    for num in num_episodes:
        print("num_episodes: ", num)
        Q = q_learning(env, num_episodes=num)
        print("Q: ", Q)
        print("Win rate: ", winrate(Q, env, num_episodes=(num // 50)))

    # Q = q_learning(env, num_episodes=50000)
    # print("Q: ", Q)
    # print("Win rate: ", winrate(Q, env, num_episodes=1000))

if __name__ == "__main__":
    main()

```

output of Q-learning

num_episodes: 500

Q: [[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]]

Win rate: 0.0

num_episodes: 1000

Q: [[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]]

Win rate: 0.0

num_episodes: 5000

Q: [[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]

[0. 0. 0. 0.]]

Win rate: 0.0

num_episodes: 10000

```

Q: [[0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]]
Win rate: 0.0
num_episodes: 50000
Q: [[0.09210638 0.0898937 0.12650789 0.09238053]
    [0.02108069 0.06834914 0.07244966 0.09018051]
    [0.04814732 0.0570449 0.05871733 0.07191822]
    [0.00765549 0.02363323 0.02308788 0.07052546]
    [0.13914253 0.05937346 0.08550687 0.03705422]
    [0. 0. 0. 0.]
    [0.01969379 0.00454257 0.06704527 0.01032517]
    [0. 0. 0. 0.]
    [0.13392048 0.09131031 0.08369002 0.25587623]
    [0.10143698 0.36239568 0.01185186 0.07745817]
    [0.10123791 0.09825144 0.05845802 0.07952147]
    [0. 0. 0. 0.]
    [0. 0. 0. 0.]
    [0.13919477 0.06862741 0.60574251 0.21020814]
    [0.65293956 0.90031118 0.51678333 0.4899492 ]
    [0. 0. 0. 0.]]
Win rate: 0.812

```

In this Q-learning implementation on the frozen environment, I observed a clear relationship between the number of training episodes and the agent's learning performance. When trained with a small number of episodes (500 to 10,000), the Q-table remained mostly unchanged with values close to zero, and the agent failed to achieve any successful episodes, resulting in a win rate of 0.0. This suggests that the agent did not explore enough or receive sufficient positive rewards to learn effective state-action values, which is particularly challenging in FrozenLake due to its stochastic nature (`is_slippery=True`). However, when the number of episodes was increased to 50,000, the Q-table began to reflect meaningful learning, especially in states closer to the goal, and the win rate significantly improved to 0.812. This demonstrates that Q-learning in such environments requires extensive exploration and a large number of episodes for the agent to converge to a useful policy. The results also highlight the importance of the ϵ -greedy strategy for maintaining exploration over time, as well as the limitations of zero initialization when not paired with sufficient training. Overall, the

experiment emphasizes the necessity of long training durations for reinforcement learning agents to succeed in highly uncertain environments like FrozenLake.